Document: draft-cheshire-dnsext-dns-sd-01.txt
Category: Standards Track
Expires 20th December 2003

Stuart Cheshire
Apple Computer, Inc.
Marc Krochmal
Apple Computer, Inc.
20th June 2003

DNS-Based Service Discovery

<draft-cheshire-dnsext-dns-sd-01.txt>

Status of this Memo

This document is an Internet-Draft and is in full conformance with all provisions of Section 10 of RFC2026. Internet-Drafts are working documents of the Internet Engineering Task Force (IETF), its areas, and its working groups. Note that other groups may also distribute working documents as Internet-Drafts.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

The list of current Internet-Drafts can be accessed at http://www.ietf.org/ietf/1id-abstracts.txt

The list of Internet-Draft Shadow Directories can be accessed at http://www.ietf.org/shadow.html

Distribution of this memo is unlimited.

Abstract

This document describes a convention for naming and structuring DNS resource records. Given a type of service that a client is looking for, and a domain in which the client is looking for that service, this convention allows clients to discover a list of named instances of that desired service, using only standard DNS queries. In short, this is referred to as DNS-based Service Discovery, or DNS-SD.

Table of Contents

1. Introduction....3
2. Conventions and Terminology Used in this Document....3
3. Design Goals....4
4. Service Instance Enumeration....5
4.1 Structured Instance Names....5
4.2 User Interface Presentation....7
4.3 Internal Handling of Names....7
4.4 What You See Is What You Get....7
4.5 Ordering of Service Instance Name Components....9
5. Service Name Resolution....11
6. Data Syntax for DNS-SD TXT Records....12
6.1 General Format Rules for DNS TXT Records....12
6.2 DNS TXT Record Format Rules for use in DNS-SD....12
6.3 DNS-SD TXT Record Size....13
6.4 Rules for Names in DNS-SD Name/Value Pairs....14
6.5 Rules for Values in DNS-SD Name/Value Pairs....15
6.6 Example TXT Record....16
6.7 Version Tag....16
7. Selective Instance Enumeration....17
8. Flagship Naming....18
9. Service Type Enumeration....19
10. Populating the DNS with Information....20
11. Relationship to Multicast DNS....20
12. Discovery of Browsing and/or Registration Domains....21
13. DNS Additional Record Generation....22
14. Comparison with Alternative Service Discovery Protocols....23
15. Real Example....25
16. IPv6 Considerations....26
17. Security Considerations....26
18. IANA Considerations....26
19. Acknowledgements....26
20. Copyright....27
21. Normative References....27
22. Informative References....28
23. Author's Addresses....29

1. Introduction

This document describes a convention for naming and structuring DNS resource records. Given a type of service that a client is looking for, and a domain in which the client is looking for that service, this convention allows clients to discover a list of named instances of a that desired service, using only standard DNS queries. In short, this is referred to as DNS-based Service Discovery, or DNS-SD.

This document proposes no change to the structure of DNS messages, and no new operation codes, response codes, resource record types, or any other new DNS protocol values. This document simply proposes a convention for how existing resource record types can be named and structured to facilitate service discovery.

This proposal is entirely compatible with today's existing unicast DNS server and client software.

This proposal is also compatible with (but not dependent on) the proposal for Multicast DNS outlined in "Performing DNS queries via IP Multicast" [mDNS].

2. Conventions and Terminology Used in this Document

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in "Key words for use in RFCs to Indicate Requirement Levels" [RFC 2119].

3. Design Goals

A good service discovery protocol needs to have many properties, three of which are mentioned below:

(i) The ability to query for services of a certain type in a certain logical domain and receive in response a list of named instances (network browsing, or "Service Instance Enumeration").

(ii) Given a particular named instance, the ability to efficiently resolve that instance name to the required information a client needs to actually use the service, i.e. IP address and port number, at the very least (Service Name Resolution).

(iii) Instance names should be relatively persistent. If a user selects their default printer from a list of available choices today, then tomorrow they should still be able to print on that printer -- even if the IP address and/or port number where the service resides have changed -- without the user (or their software) having to repeat the network browsing step a second time.

In addition, if it is to become successful, a service discovery protocol should be so simple to implement that virtually any device capable of implementing IP should not have any trouble implementing the service discovery software as well.

These goals are discussed in more detail in the remainder of this document. A more thorough treatment of service discovery requirements may be found in "Requirements for the Replacement of AppleTalk Name Binding Protocol" [NBP]. That document draws upon examples from a decade-and-a-half of operational experience with AppleTalk Name Binding Protocol to develop a list of universal requirements which are broadly applicable to any potential service discovery protocol.

4. Service Instance Enumeration

DNS SRV records [RFC 2782] are useful for locating instances of a particular type of service when all the instances are effectively indistinguishable and provide the same service to the client.

For example, SRV records with the (hypothetical) name "_http._tcp.example.com." would allow a client to discover a list of all servers implementing the "_http._tcp" service (i.e. Web servers) for the "example.com." domain. The unstated assumption is that all these servers offer an identical set of Web pages, and it doesn't matter to the client which of the servers it uses, as long as it selects one at random according to the weight and priority rules laid out in RFC 2782.

Instances of other kinds of service are less easily interchangeable. If a word processing application were to look up the (hypothetical) SRV record "_ipp._tcp.example.com." to find the list of IPP printers at Example Co., then picking one at random and printing on it would probably not be what the user wanted.

The remainder of this section describes how SRV records may be used in a slightly different way to allow a user to discover the names of all available instances of a given type of service, in order to select the particular instance the user desires.

4.1 Structured Instance Names

This document borrows the logical service naming syntax and semantics from DNS SRV records, but adds one level of indirection. Instead of requesting records of type "SRV" with name "_ipp._tcp.example.com.", the client requests records of type "PTR" (pointer from one name to another in the DNS namespace).

In effect, if one thinks of the domain name "_ipp._tcp.example.com." as being analogous to an absolute path to a directory in a file system then the PTR lookup is akin to performing a listing of that directory to find all the files it contains. (Remember that domain names are expressed in reverse order compared to path names: An absolute path name is read from left to right, beginning with a leading slash on the left, and then the top level directory, then the next level directory, and so on. A fully-qualified domain name is read from right to left, beginning with the dot on the right -- the

root label -- and then the top level domain to the left of that, and the second level domain to the left of that, and so on. If the fully-qualified domain name "_ipp._tcp.example.com." were expressed as a file system path name, it would be "/com/example/_tcp/_ipp".)

The result of this PTR lookup for the name "<Service>.<Domain>" is a list of zero or more PTR records giving Service Instance Names of the form:

Service Instance Name = <Instance> . <Service> . <Domain>

The <Instance> portion of the Service Instance Name is a single DNS label, containing arbitrary UTF-8-encoded text [RFC 2279]. It is a user-friendly name, meaning that it is allowed to contain any characters, without restriction, including spaces, upper case, lower case, punctuation -- including dots -- accented characters, non-roman text, and anything else that may be represented using UTF-8. DNS recommends guidelines for allowable characters for host names [RFC 1033][RFC 1034][RFC 1035], but Service Instance Names are not host names. Service Instance Names are not intended to ever be typed in by a normal user; the user selects a Service Instance Name by selecting it from a list of choices presented on the screen.

Note that just because this protocol supports arbitrary UTF-8-encoded names doesn't mean that any particular user or administrator is obliged to make use of that capability. Any user is free, if they wish, to continue naming their services using only letters, digits and hyphens, with no spaces, capital letters, or other punctuation.

DNS labels are currently limited to 63 octets in length. UTF-8 encoding can require up to four octets per Unicode character, which means that in the worst case, the <Instance> portion of a name could be limited to fifteen Unicode characters. However, the Unicode characters with longer UTF-8 encodings tend to be the more obscure ones, and tend to be the ones that convey greater meaning per character.

Note that any character in the commonly-used 16-bit Unicode space can be encoded with no more than three octets of UTF-8 encoding. This means that an Instance name can contain up to 21 Kanji characters, which is a sufficiently expressive name for most purposes.

The <Service> portion of the Service Instance Name consists of a pair of DNS labels, following the established convention for SRV records [RFC 2782]. The first label of the service pair is the application protocol name, as recorded in the IANA list of assigned application protocol names and port numbers [ports]. The second label of the service pair is either "_tcp" or "_udp", depending on the transport protocol used by the application.

The <Domain> portion of the Service Instance Name is a conventional DNS domain name, consisting of as many labels as appropriate. For example, "apple.com.", "cs.stanford.edu.", and "eng.us.ibm.com." are all valid domain names for the <Domain> portion of the Service Instance Name.

4.2 User Interface Presentation

The names resulting from the PTR lookup are presented to the user in a list for the user to select one (or more). Typically only the first label is shown (the user-friendly <Instance> portion of the name). The <Service> and <Domain> are already known to the user, these having been provided by the user in the first place, by the act of indicating the service being sought, and the domain in which to look for it.

Having chosen the desired named instance, the Service Instance Name may then be used immediately, or saved away in some persistent user-preference data structure for future use, depending on what is appropriate for the application in question.

4.3 Internal Handling of Names

If the <Instance>, <Service> and <Domain> portions are internally concatenated together into a single string, then care must be taken with the <Instance> portion, since it is allowed to contain any characters, including dots.

Any dots in the <Instance> portion should be escaped by preceeding them with a backslash ("." becomes "\."). Likewise, any backslashes in the <Instance> portion should also be escaped by preceeding them with a backslash ("\" becomes "\\"). Having done this, the three components of the name may be safely concatenated. The backslash-escaping allows literal dots in the name (escaped) to be distinguished from label-separator dots (not escaped).

The resulting concatenated string may be safely passed to standard DNS APIs like res_query(), which will interpret the string correctly provided it has been escaped correctly, as described here.

4.4 What You See Is What You Get

Some service discovery protocols decouple the true service identifier from the name presented to the user. The true service identifier used by the protocol is an opaque unique id, often represented using a long string of hexadecimal digits, and should never be seen by the typical user. The name presented to the user is merely one of the ephemeral attributes attached to this opaque identifier.

The problem with this approach is that it decouples user perception from reality:

* What happens if there are two service instances, with different unique ids, but they have inadvertently been given the same

user-visible name? If two instances appear in an on-screen list with the same name, how does the user know which is which?

* Suppose a printer breaks down, and the user replaces it with another printer of the same make and model, and configures the new printer with the exact same name as the one being replaced: "Stuart's Printer". Now, when the user tries to print, the on-screen print dialog tells them that their selected default printer is "Stuart's Printer". When they browse the network to see what is there, they see a printer called "Stuart's Printer", yet when the user tries to print, they are told that the printer "Stuart's Printer" can't be found. The hidden internal unique id that the software is trying to find on the network doesn't match the hidden internal unique id of the new printer, even though its apparent "name" and its logical purpose for being there are the same. To remedy this, the user typically has to delete the print queue they have created, and then create a new (apparently identical) queue for the new printer, so that the new queue will contain the right hidden internal unique id. Having all this hidden information that the user can't see makes for a confusing and frustrating user experience, and exposing long ugly hexadecimal strings to the user and forcing them to understand what they mean is even worse.

* Suppose an existing printer is moved to a new department, and given a new name and a new function. Changing the user-visible name of that piece of hardware doesn't change its hidden internal unique id. Users who had previously created print queues for that printer will still be accessing the same hardware by its unique id, even though the logical service that used to be offered by that hardware has ceased to exist.

To solve these problems requires the user or administrator to be aware of the supposedly hidden unique id, and to set its value correctly as hardware is moved around, repurposed, or replaced, thereby contradicting the notion that it is a hidden identifier that human users never need to deal with. Requiring the user to unserstand this expert behind-the-scenes knowledge of what is *really* going on is just one more burden placed on the user when they are trying to diagnose why their computers and network devices are not working as expected.

These anomalies and counter-intuitive behaviours can be eliminated by maintaining a tight bidirectional one-to-one mapping between what the

user sees on the screen and what is really happening "behind the curtain". If something is configured incorrectly, then that is apparent in the familiar day-to-day user interface that everyone understands, not in some little-known rarely-used "expert" interface.

In summary: The user-visible name is the primary identifier for a service. If the user-visible name is changed, then conceptually the

service being offered is a different service -- even if the hardware in question has been recycled from some other part of the organization where it was previously in use providing the same type of service to a different group of people, or for a different purpose. If the user-visible name stays the same, then conceptually the service being offered is the same service -- even if the hardware in question is new hardware brought in to replace some old equipment that was broken, worn out, or out-of-date.

4.5 Ordering of Service Instance Name Components

There have been questions about why services are named using DNS Service Instance Names of the form:

Service Instance Name = <Instance> . <Service> . <Domain>

instead of:

Service Instance Name = <Service> . <Instance> . <Domain>

There are three reasons why it is beneficial to name service instances with the parent domain as the most-significant (rightmost) part of the name, then the abstract service type as the nextmost significant, and then the specific instance name as the least-significant (leftmost) part of the name:

4.5.1. Semantic Structure

The facility being provided by browsing ("Service Instance Enumeration") is effectively enumerating the leaves of a tree structure. A given domain offers zero or more services. For each of those service types, there may be zero or more instances of that service.

The user knows what type of service they are seeking. (If they are running an FTP client, they are looking for FTP servers. If they have a document to print, they are looking for entities that speak some known printing protocol.) The user knows in which organizational or geographical domain they wish to search. (The user does not want a single flat list of every single printer on the planet, even if such a thing were possible.) What the user does not know in advance is whether the service they seek is

offered in the given domain, or if so, how many instances are offered, and the names of those instances. Hence having the instance names be the leaves of the tree is consistent with this semantic model.

Having the service types be the terminal leaves of the tree would imply that the user knows the domain name, and already knows the

name of the service instance, but doesn't have any idea what the service does. Clearly this is a less useful model.

4.5.2. Network Efficiency

When a DNS response contains multiple answers, name compression works more effectively if all the names contain a common suffix. If many answers in the packet have the same <Service> and <Domain>, then each occurrence of a Service Instance Name can be expressed using only the <Instance> part followed by a two-byte compression pointer referencing a previous appearance of "<Service>.<Domain>". This efficiency would not be possible if the <Service> component appeared first in each name.

4.5.3. Operational Flexibility

This name structure allows subdomains to be delegated along logical service boundaries. For example, the network administrator at Example Co. could choose to delegate the "_tcp.example.com." subdomain to a different machine, so that the machine handling service discovery doesn't have to be the same as the machine handling other day-to-day DNS operations. (It *can* be the same machine if the administrator so chooses, but the point is that the administrator is free to make that choice.) Furthermore, if the network administrator wishes to delegate all information related to IPP printers to a machine dedicated to that specific task, this is easily done by delegating the "_ipp._tcp.example.com." subdomain to the desired machine. It is also convenient to set security policies on a per-zone/per-subdomain basis. For example, the administrator may choose to enable DNS Dynamic Update [RFC 2136] [RFC 3007] for printers registering in the "_ipp._tcp.example.com." subdomain, but not for other zones/subdomains. This easy flexibility would not exist if the <Service> component appeared first in each name.

5. Service Name Resolution

Given a particular Service Instance Name, when a client needs to contact that service, it sends a DNS request for the SRV record of that name.

The result of the DNS request is a SRV record giving the port number and target host where the service may be found.

The use of SRV records is very important. There are only 65535 TCP port numbers available. These port numbers are being allocated one-per-application-protocol at an alarming rate. Some protocols like the X Window System have a block of 64 TCP ports allocated (6000-6063). If we start allocating blocks of 64 TCP ports at a time, we will run out even faster. Using a different TCP port for each different instance of a given service on a given machine is entirely sensible, but allocating large static ranges, as was done for X, is a very inefficient way to manage a limited resource. On any given host, most TCP ports are reserved for services that will never run on that particular host. This is very poor utilization of the limited port space. Using SRV records allows each host to allocate its available port numbers dynamically to those services running on that host that need them, and then advertise the allocated port numbers via SRV records. Allocating the available listening port numbers locally on a per-host basis as needed allows much better utilization of the available port space than today's centralized global allocation.

In some environments there may be no compelling reason to assign managed names to every host, since every available service is accessible by name anyway, as a first-class entity in its own right. However, the DNS packet format and record format still require a host name to link the target host referenced in the SRV record to the address records giving the IPv4 and/or IPv6 addresses for that hardware. In the case where no natural host name is available, the SRV record may give its own name as the name of the target host, and then the requisite address records may be attached to that same name. It is perfectly permissible for a single name in the DNS hierarchy to have multiple records of different type attached. (The only restriction being that a given name may not have both a CNAME record and other records at the same time.)

In the event that more than one SRV is returned, clients MUST correctly interpret the priority and weight fields -- i.e. Lower numbered priority servers should be used in preference to higher

numbered priority servers, and servers with equal priority should be selected randomly in proportion to their relative weights.

6. Data Syntax for DNS-SD TXT Records

Some services discovered via Service Instance Enumeration may need more than just an IP address and port number to properly identify the service. For example, printing via the LPR protocol often specifies a queue name. This queue name is typically short and cryptic, and need not be shown to the user. It should be regarded the same way as the IP address and port number -- it is one component of the addressing information required to identify a specific instance of a service being offered by some piece of hardware. Similarly, a file server may have multiple volumes, each identified by its own volume name. A Web server typically has multiple pages, each identified by its own URL. In these cases, the necessary additional data is stored in a TXT record with the same name as the SRV record. The specific nature of that additional data, and how it is to be used, is service-dependent, but the overall syntax of the data in the TXT record is standardized, as described below.

6.1 General Format Rules for DNS TXT Records

A DNS TXT record can be up to 65535 (0xFFFF) bytes long. The total length is indicated by the length given in the resource record header in the DNS message. There is no way to tell directly from the data alone how long it is (e.g. there is no length count at the start, or terminating NULL byte at the end).

The format of the data within a DNS TXT record is zero or more strings, packed together in memory without any intervening gaps or padding bytes for word alignment.

The format of each constituent string within the DNS TXT record is a single length byte, followed by 0-255 bytes of text data.

These format rules are defined in Section 3.3.14 of RFC 1035, and are not specific to DNS-SD. DNS-SD simply specifies a usage convention for what data should be stored in those constituent strings.

6.2 DNS TXT Record Format Rules for use in DNS-SD

DNS-SD uses DNS TXT records to store arbitrary name/value pairs conveying additional information about the named service. Each name/value pair is encoded as its own constituent string within the

DNS TXT record, in the form "name=value". Everything up to the first '=' character is the name. Everything after the first '=' character to the end of the string (including subsequent '=' characters, if any) is the value. Specific rules governing names and values are given below. Each author defining a DNS-SD profile for discovering instances of a particular type of service should define the base set of name/value attributes that are valid for that type of service.

Using this standardized name/value syntax within the TXT record makes it easier for these base definitions to be expanded later by defining additional named attributes. If an implementation sees unknown attribute names in a service TXT record, it SHOULD silently ignore them.

The TCP (or UDP) port number of the service, and target host name, are given in the SRV record. This information -- target host name and port number -- MUST NOT be duplicated using name/value attributes in the TXT record.

The intention of DNS-SD TXT records is to convey a small amount of useful additional information about a service. Ideally it SHOULD NOT be necessary for a client to retrieve this additional information before it can usefully establish a connection to the service. For a well-designed TCP-based application protocol, it should be possible, knowing only the host name and port number, to open a connection to that listening process, and then perform version- or feature-negotiation to determine the capabilities of the service instance. For example, when connecting to an AppleShare server over TCP, the client enters into a protocol exchange with the server to determine which version of the AppleShare protocol the server implements, and which optional features or capabilities (if any) are available. For a well-designed application protocol, clients should be able to connect and use the service even if there is no information at all in the TXT record. In this case, the information in the TXT record should be viewed as a performance optimization -- when a client discovers many instances of a service, the TXT record allows the client to know some rudimentary information about each instance without having to open a TCP connection to each one and interrogate every service instance separately. Extreme care should be taken when doing this to ensure that the information in the TXT record is in agreement with the information retrieved by a client connecting over TCP.

There are legacy protocols which provide no feature negotiation capability, and in these cases it may be useful to convey necessary information in the TXT record. For example, when printing using the old Unix LPR (port 515) protocol, the LPR service provides no way for the client to determine whether a particular printer accepts PostScript, or what version of PostScript, etc. In this case it is appropriate to embed this information in the TXT record, because the alternative is worse -- passing around written instructions to the users, arcane manual configuration of "/etc/printcap" files, etc.

6.3 DNS-SD TXT Record Size

The total size of a typical DNS-SD TXT record is intended to be small -- 100 bytes or less.

In cases where more data is justified (e.g. LPR printing), keeping

the total size under 400 bytes should allow it to fit in a single standard 512-byte DNS message. (This standard DNS message size is defined in RFC 1035.)

In extreme cases where even this is not enough, keeping the size of the TXT record under 1300 bytes should allow it to fit in a single 1500-byte Ethernet packet.

Using TXT records larger than 1300 bytes is NOT RECOMMENDED at this time.

6.4 Rules for Names in DNS-SD Name/Value Pairs

The "Name" MUST be at least one character. Strings beginning with an '=' character (i.e. the name is missing) SHOULD be silently ignored.

The characters of "Name" MUST be printable US-ASCII values (0x20-0x7E), excluding '=' (0x3D).

Spaces in the name are significant, whether leading, trailing, or in the middle -- so don't include any spaces unless you really intend that!

Case is ignored when interpreting a name, so "papersize=A4", "PAPERSIZE=A4" and "Papersize=A4" are all identical.

If there is no '=', then it is a boolean attribute, and is simply identified as being present, with no value.

When examining a TXT record for a given named attribute, there are therefore four broad categories of results which may be returned:

* Attribute not present (Absent)

* Attribute present, with no value
  (e.g. "Anon Allowed" -- server allows anonymous connections)

* Attribute present, with empty value (e.g. "Installed PlugIns=" -- server supports plugins, but none are presently installed)

* Attribute present, with non-empty value
  (e.g. "Installed PlugIns=JPEG,MPEG2,MPEG4")

Unless specified otherwise by a particular DNS-SD profile, a given attribute name may appear at most once in a TXT record. If a client receives a TXT record containing the same attribute name more than once, then the client SHOULD silently ignore all but the first occurrence of that attribute. For client implementations that process a DNS-SD TXT record from start to end, placing name/value pairs into a hash table, using the name as the hash table key, this means that

if the implementation attempts to add a new name/value pair into the table and finds an entry with the same name already present, then the new entry being added should be silently discarded instead. For client implementations that retrieve name/value pairs by searching the TXT record for the requested name, they should search the TXT record from the start, and simply return the first matching name they find.

Each author defining a DNS-SD profile for discovering instances of a particular type of service should define the interpretation of these different kinds of result. For example, for some keys, there may be a natural boolean interpretation:

* Absent implies 'false'
* Present with no value implies 'true'

For other keys it may be sensible to define other semantics, such as:

* Present with value implies that value.
  E.g. "Color=4" for a four-color ink-jet printer,
  or "Color=6" for a six-color ink-jet printer.

* Present with empty value implies 'false'. E.g. Not a color printer.

* Absent implies 'Unknown'. E.g. A print server connected to some unknown printer where the print server doesn't actually know if the printer does color or not -- which gives a very bad user experience and should be avoided wherever possible.

(Note that this is a hypothetical example, not an example of real name/value keys for printing.)

As a general rule, attribute names that contain no dots are defined as part of the open-standard definition written by the person or group defining the DNS-SD profile for discovering that particular service type. Vendor-specific extensions should be given names of the form "keyname.company.com=value", using a domain name legitimately registered to the person or organization creating the vendor-specific key. This reduces the risk of accidental conflict if different organizations each define their own vendor-specific keys.

6.5 Rules for Values in DNS-SD Name/Value Pairs

If there is an '=', then everything after the first '=' to the end of the string is the value. The value can contain any eight-bit values including '='. Leading or trailing spaces are part of the value, so don't put them there unless you intend them to be there. Any quotation marks around the value are part of the value, so don't put them there unless you intend them to be part of the value.

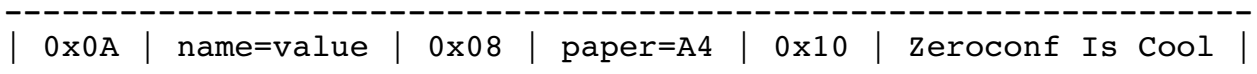
The value is opaque binary data. Often the value for a particular attribute will be US-ASCII (or UTF-8) text, but it is legal for a value to be any binary data. For example, if the value of a key is an IPv4 address, that address should simply be stored as four bytes of binary data, not as a variable-length 7-15 byte ASCII string giving the address represented in textual dotted decimal notation.

Generic debugging tools should generally display all attribute values as a hex dump, with accompanying text alongside displaying the UTF-8 interpretation of those bytes, except for attributes where the debugging tool has embedded knowledge that the value is some other kind of data.

Authors defining DNS-SD profiles SHOULD NOT convert binary attribute data types into printable text (e.g. using hexadecimal, Base64 or UU encoding) merely for the sake of making the data be printable text when seen in a generic debugging tool. Doing this simply bloats the size of the TXT record, without truly making the data any more understandable to someone looking at it in a generic debugging tool.

6.6 Example TXT Record

The TXT record below contains three syntactically valid name/value pairs. (The meaning of these name/value pairs, if any, would depend on the definitions pertaining to the service in question that is using them.)



6.7 Version Tag

It is recommended that authors defining DNS-SD profiles include an attribute of the form "txtvers=xxx" in their definition, and require it to be the first name/value pair in the TXT record. This information in the TXT record can be useful to help clients maintain backwards compatibility with older implementations if it becomes necessary to change or update the specification over time. Even if the profile author doesn't anticipate the need for any future incompatible changes, having a version number in the specification provides useful insurance should incompatible changes become

unavoidable. Clients SHOULD ignore TXT records with a txtvers number higher (or lower) than the version(s) they know how to interpret.

Note that the version number in the txtvers tag describes the version of the TXT record specification being used to create this TXT record, not the version of the application protocol that will be used if the client subsequently decides to contact that service. Ideally, every

DNS-SD TXT record specification starts at txtvers=1 and stays that way forever. Improvements can be made by defining new keys that older clients silently ignore. The only reason to increment the version number is if the old specification is subsequently found to be so horribly broken that there's no way to do a compatible forward revision, so the txtvers number has to be incremented to tell all the old clients they should just not even try to understand this new TXT record.

If there is a need to indicate which version number(s) of the application protocol the service implements, the recommended key name for this is "protovers".

7. Selective Instance Enumeration

This document does not attempt to define an arbitrary query language for service discovery, nor do we believe one is necessary.

However, there are some circumstances where narrowing the list of results may be useful. A Web browser client that is able to retrieve HTML documents via HTTP and display them may also be able to retrieve HTML documents via FTP and display them, but only in the case of FTP servers that allow anonymous login. For that Web browser, discovering all FTP servers on the network is not useful. The Web browser only wants to discover FTP servers that it is able to talk to. In this case, a subtype of "_ftp._tcp" could be defined. Instead of issuing a query for "_ftp._tcp.<Domain>", the Web browser issues a query for "_anon._ftp._tcp.<Domain>", where "_anon" is a defined subtype of "_ftp._tcp". The response to this query only includes the names of SRV records for FTP servers that are willing to allow anonymous login.

Note that the FTP server's Service Instance Name is unchanged -- it is still something of the form "The Server._ftp._tcp.example.com." The subdomain in which FTP server SRV records are registered defines the namespace within which FTP server names are unique. Additional subtypes (e.g. "_anon") of the basic service type (e.g. "_ftp._tcp") serve to narrow the list of results, not to create more namespace.

As for the TXT record name/value pairs, the list of possible subtypes, if any, are defined and specified separately for each basic service type.

8. Flagship Naming

In some cases, there may be several network protocols available which all perform roughly the same logical function. For example, the printing world has the LPR protocol, and the Internet Printing Protocol (IPP), both of which cause printed sheets to be emitted from printers in much the same way. In addition, many printer vendors send their own proprietary page description language (PDL) data over a TCP connection to TCP port 9100, herein referred to as the "pdl-datastream" protocol. In an ideal world we would have only one network printing protocol, and it would be sufficiently good that no one felt a compelling need to invent a different one. However, in practice, multiple legacy protocols do exist, and a service discovery protocol has to accommodate that.

Many printers implement all three printing protocols: LPR, IPP, and pdl-datastream. For the benefit of clients that may speak only one of those protocols, all three are advertised.

However, some clients may implement two, or all three of those printing protocols. When a client looks for all three service types on the network, it will find three distinct services -- an LPR service, an IPP service, and a pdl-datastream service -- all of which cause printed sheets to be emitted from the same physical printer.

In the case of multiple protocols like this that all perform effectively the same function, the client should suppress duplicate names and display each name only once. When the user prints to a given named printer, the printing client is responsible for choosing the protocol which will best achieve the desired effect, without, for example, requiring the user to make a manual choice between LPR and IPP.

As described so far, this all works very well. However, consider some future printer that only supports IPP printing, and some other future printer that only supports pdl-datastream printing. The name spaces for different service types are intentionally disjoint -- it is acceptable and desirable to be able to have both a file server called "Sales Department" and a printer called "Sales Department". However, it is not desirable, in the common case, to have two different printers both called "Sales Department", even if those printers are implementing different protocols.

To help guard against this, when there are two or more network

protocols which perform roughly the same logical function, one of the protocols is declared the "flagship" of the fleet of related protocols. Typically the flagship protocol is the oldest and/or best-known protocol of the set.

If a device does not implement the flagship protocol, then it instead creates an empty SRV record (priority=0, weight=0, port=0, target

host = null) with that name. If, when it attempts to create this SRV record, it finds that one with that name already exists, then it knows that this name is already taken by some entity implementing one of the protocols from the class, and it must choose another. If no SRV record already exists, then the act of creating it stakes a claim to that name so that future devices in the same class will not try to use it.

By defining a common well-known flagship protocol for the class, future devices that may not even know about each other's protocols have a common ground where they can coordinate to verify uniqueness of names.

No PTR record is created advertising the presence of empty flagship SRV records, since they do not represent a real service being advertised.

9. Service Type Enumeration

In general, clients are not interested in finding *every* service on the network, just the services that the client knows how to talk to. (Software designers may *think* there's some value to finding *every* service on the network, but that's just wooly thinking.)

However, for problem diagnosis and network management tools, it may be useful for network administrators to find the list of advertised service types on the network, even if those service names are just opaque identifiers and not particularly informative in isolation.

For this reason, a special meta-query is defined. A DNS query for PTR records with the name "_services._mdns._udp.<Domain>" yields a list of PTR records, where the rdata of each PTR record is the name of a service type. A subsequent query for PTR records with one of those names yields a list of instances of that service type.

10. Populating the DNS with Information

How the SRV and PTR records that describe services and allow them to be enumerated make their way into the DNS is outside the scope of this document. However, it can happen easily in any of a number of ways, for example:

On some networks, the administrator might manually enter the records into the name server's configuration file.

A network monitoring tool could output a standard zone file to be read into a conventional DNS server. For example, a tool that can find Apple LaserWriters using AppleTalk NBP could find the list of printers, communicate with each one to find its IP address, PostScript version, installed options, etc., and then write out a DNS zone file describing those printers and their capabilities using DNS resource records. That information would then be available to DNS-SD clients that don't implement AppleTalk NBP, and don't want to.

Future IP printers could use Dynamic DNS Update [RFC 2136] to automatically register their own SRV and PTR records with the DNS server.

A printer manager device which has knowledge of printers on the network through some other management protocol could also use Dynamic DNS Update [RFC 2136].

Alternatively, a printer manager device could implement enough of the DNS protocol that it is able to answer DNS requests directly, and Example Co.'s main DNS server could delegate the _ipp._tcp.example.com subdomain to the printer manager device.

Zeroconf printers answer Multicast DNS requests on the local link for appropriate PTR and SRV names ending with ".local." [mDNS].

11. Relationship to Multicast DNS

DNS-Based Service Discovery is not strictly related to Multicast DNS, but the two are highly complementary, particularly in Zeroconf environments [ZC].

Lookups for PTR records of the form "<Service>.local." are defined to use multicast, and return a list of named instances of the form

"<Instance>.<Service>.local."

12. Discovery of Browsing and/or Registration Domains

One of the main reasons for DNS-Based Service Discovery is so that when a visiting client (e.g. a laptop computer) arrives at a new network, it can discover what services are available on that network without manual configuration. This logic that applies to discovering services without manual configuration also applies to discovering the domains in which services are registered without requiring manual configuration.

This discovery is performed recursively, using Unicast or Multicast DNS. Four special RR names are reserved for this purpose:

```
                _browse._mdns._udp.<domain>
      _default._browse._mdns._udp.<domain>
             _register._mdns._udp.<domain>
    _default._register._mdns._udp.<domain>
```

By performing PTR queries for these names, a client can learn, respectively:

- A list of domains recommended for browsing
- A single recommended default domain for browsing
- A list of domains recommended for registering services using Dynamic Update
- A single recommended default domain for registering services.

These domains are purely advisory. The client or user is free to browse and/or register services in any domains. The purpose of these special queries is to allow software to create a user-interface that displays a useful list of suggested choices to the user, from which they may make a suitable selection, or ignore the offered suggestions and manually enter their own choice.

The <domain> part of the name may be ".local." (meaning "perform the query using link-local multicast) or it may be learned through some other mechanism, such as the DHCP "Domain" option (option code 15) [RFC 2132] or the DHCP "Domain Search" option (option code 119) [RFC 3397]. Sophisticated clients may perform these queries both in ".local." and in one or more unicast domains, and then present the user with an aggregate result, combining the information received

from all sources.

13. DNS Additional Record Generation

DNS has an efficiency feature whereby a DNS server may place additional records in the Additional Section of the DNS Message. These additional records are typically records that the client did not explicitly request, but the server has reasonable grounds to expect that the client might request them shortly.

This section recommends which additional records should be generated to improve network efficiency for both unicast and multicast DNS-SD responses.

13.1 DNS-SD PTR Queries

In response to a DNS-SD PTR query, the server/responder SHOULD include the following additional records:

- The SRV record(s) named in the PTR rdata.
- The TXT record(s) named in the PTR rdata.
- All address records (type "A" and "AAAA") named in the SRV rdata.

13.2 DNS-SD SRV Queries

In response to a DNS-SD SRV query, the server/responder SHOULD include the following additional records:

- All address records (type "A" and "AAAA") named in the SRV rdata.

13.3 DNS-SD TXT Queries

In response to a DNS-SD TXT query, no additional records are required.

13.4 Other Queries

In response to address queries, or other record types, no additional records are required by this document.

14. Comparison with Alternative Service Discovery Protocols

Over the years there have been many proposed ways to do network service discovery with IP, but none achieved ubiquity in the marketplace. Certainly none has achieved anything close to the ubiquity of today's deployment of DNS servers, clients, and other infrastructure.

The advantage of using DNS as the basis for service discovery is that it makes use of those existing servers, clients, protocols, infrastructure, and expertise. Existing network analyser tools already know how to decode and display DNS packets for network debugging.

For ad-hoc networks such as Zeroconf environments, peer-to-peer multicast protocols are appropriate. It is almost certain that the Zeroconf host profile [ZCHP] will specify the use of a DNS-like protocol over IP Multicast for host name resolution in the absence of DNS servers. Given that Zeroconf hosts will have to implement this Multicast-based DNS-like protocol anyway, it makes sense for them to also perform service discovery using that same Multicast-based DNS-like software, instead of also having to implement an entirely different service discovery protocol.

In larger networks, a high volume of enterprise-wide IP multicast traffic may not be desirable, so any credible service discovery protocol intended for larger networks has to provide some facility to aggregate registrations and lookups at a central server (or servers) instead of working exclusively using multicast. This requires some service discovery aggregation server software to be written, debugged, deployed, and maintained. This also requires some service discovery registration protocol to be implemented and deployed for clients to register with the central aggregation server. Virtually every company with an IP network already runs a DNS server, and DNS already has a dynamic registration protocol [RFC 2136]. Given that virtually every company already has to operate and maintain a DNS server anyway, it makes sense to take advantage of this instead of also having to learn, operate and maintain a different service registration server. It should be stressed again that using the same software and protocols doesn't necessarily mean using the same physical piece of hardware. The DNS-SD service discovery functions do not have to be provided by the same piece of hardware that is currently providing the company's DNS name service. The "_tcp.<Domain>" subdomain may be delegated to a different piece of

hardware. However, even when the DNS-SD service is being provided by a different piece of hardware, it is still the same familiar DNS server software that is running, with the same configuration file syntax, the same log file format, and so forth.

Service discovery needs to be able to provide appropriate security. DNS already has existing mechanisms for security [RFC 2535].

In summary:

Service discovery requires a central aggregation server.
DNS already has one: It's called a DNS server.

Service discovery requires a service registration protocol.
DNS already has one: It's called DNS Dynamic Update.

Service discovery requires a query protocol
DNS already has one: It's called DNS.

Service discovery requires security mechanisms.
DNS already has security mechanisms: DNSSEC.

Service discovery requires a multicast mode for ad-hoc networks.
DNS doesn't have one right now, but it will soon, to meet Zeroconf requirements.

It makes more sense to use the existing software that every network needs already, instead of deploying an entire parallel system just for service discovery.

15. Real Example

The following examples were prepared using standard unmodified nslookup and standard unmodified BIND running on GNU/Linux.

Note: In real products, this information is obtained and presented to the user using graphical network browser software, not command-line tools, but if you wish you can try these examples for yourself as you read along, using the command-line tools already available on your own Unix machine.

15.1 Question: What FTP servers are being advertised from dns-sd.org?

```
nslookup -q=ptr _ftp._tcp.dns-sd.org.
_ftp._tcp.dns-sd.org name=Apple\032QuickTime\032Files.dns-sd.org
_ftp._tcp.dns-sd.org name=Microsoft\032Developer\032Files.dns-sd.org
_ftp._tcp.dns-sd.org name=Registered\032Users'\032Only.dns-sd.org
```

Answer: There are three, called "Apple QuickTime Files", "Microsoft Developer Files" and "Registered Users' Only".

Note that nslookup escapes spaces as "\032" for display purposes, but a graphical DNS-SD browser does not.

15.2 Question: What FTP servers allow anonymous access?

```
nslookup -q=ptr _anon._ftp._tcp.dns-sd.org
_anon._ftp._tcp.dns-sd.org
                      name=Apple\032QuickTime\032Files.dns-sd.org
_anon._ftp._tcp.dns-sd.org
                      name=Microsoft\032Developer\032Files.dns-sd.org
```

Answer: Only "Apple QuickTime Files" and "Microsoft Developer Files" allow anonymous access.

15.3 Question: How do I access "Apple QuickTime Files"?

```
nslookup -q=any "Apple\032QuickTime\032Files.dns-sd.org."
Apple\032QuickTime\032Files.dns-sd.org  text = "path=/quicktime"
Apple\032QuickTime\032Files.dns-sd.org
     priority = 0, weight = 0, port= 21 host = ftp.apple.com
```

```
ftp.apple.com   internet address = 17.254.0.27
ftp.apple.com   internet address = 17.254.0.31
ftp.apple.com   internet address = 17.254.0.26
```

Answer: You need to connect to ftp.apple.com, port 21, path "/quicktime". The addresses for ftp.apple.com are also given.

16. IPv6 Considerations

IPv6 has no significant differences, except that the address of the SRV record's target host is given by the appropriate IPv6 address records instead of the IPv4 "A" record.

17. Security Considerations

DNSSEC [RFC 2535] should be used where the authenticity of information is important. Since DNS-SD is just a naming and usage convention for records in the existing DNS system, it has no specific additional security requirements over and above those that already apply to DNS queries and DNS updates.

18. IANA Considerations

The IANA will continue allocating symbolic service/protocol names, just as they do today every time someone requests a TCP or UDP port number. In future, as more applications start using DNS SRV records, it may make sense for IANA to start allocating symbolic service/protocol names without an associated hard-coded port number.

The textual nature of service/protocol names means that there are almost infinitely many more of them available than the finite set of 65535 possible port numbers. This means that developers can produce experimental implementations using unregistered service names with little chance of accidental collision, providing service names are chosen with appropriate care. However, this document strongly advocates that on or before the date a product ships, developers should register their service names with IANA.

Some developers have expressed concern that publicly registering their service names (and port numbers today) with IANA before a product ships may give away clues about that product to their competition. For this reason, IANA should consider allowing service name applications to remain secret for some period of time, much as US patent applications remain secret for two years after the date of filing.

19. Acknowledgements

We would like to thank (in alphabetical order) Richard Brown, Josh Graessley, Erik Guttman, Paul Vixie, and Bill Woodcock, for their contributions.

20. Copyright

Copyright (C) The Internet Society June 2003.
All Rights Reserved.

This document and translations of it may be copied and furnished to others, and derivative works that comment on or otherwise explain it or assist in its implementation may be prepared, copied, published and distributed, in whole or in part, without restriction of any kind, provided that the above copyright notice and this paragraph are included on all such copies and derivative works. However, this document itself may not be modified in any way, such as by removing the copyright notice or references to the Internet Society or other Internet organizations, except as needed for the purpose of developing Internet standards in which case the procedures for copyrights defined in the Internet Standards process must be followed, or as required to translate it into languages other than English.

The limited permissions granted above are perpetual and will not be revoked by the Internet Society or its successors or assigns.

This document and the information contained herein is provided on an "AS IS" basis and THE INTERNET SOCIETY AND THE INTERNET ENGINEERING TASK FORCE DISCLAIMS ALL WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTY THAT THE USE OF THE INFORMATION HEREIN WILL NOT INFRINGE ANY RIGHTS OR ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE.

21. Normative References

[ports] IANA list of assigned application protocol names and port numbers <http://www.iana.org/assignments/port-numbers>

[RFC 1033] Lottor, M., "Domain Administrators Operations Guide", RFC 1033, November 1987.

[RFC 1034] Mockapetris, P., "Domain Names - Concepts and Facilities", STD 13, RFC 1034, November 1987.

[RFC 1035] Mockapetris, P., "Domain Names - Implementation and Specifications", STD 13, RFC 1035, November 1987.

[RFC 2119] Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", RFC 2119, March 1997.

[RFC 2279] Yergeau, F., "UTF-8, a transformation format of ISO 10646", RFC 2279, January 1998.

[RFC 2782] Gulbrandsen, A., et al., "A DNS RR for specifying the location of services (DNS SRV)", RFC 2782, February 2000.

22. Informative References

[mDNS] Cheshire, S., "Performing DNS queries via IP Multicast", Internet-Draft (work in progress), draft-cheshire-dnsext-multicastdns-02.txt, June 2003.

[NBP] Cheshire, S., "Requirements for the Replacement of AppleTalk Name Binding Protocol", Internet-Draft (work in progress), draft-cheshire-dnsext-nbp-02.txt, June 2003.

[RFC 2132] Alexander, S., and Droms, R., "DHCP Options and BOOTP Vendor Extensions", RFC 2132, March 1997.

[RFC 2136] Vixie, P., et al., "Dynamic Updates in the Domain Name System (DNS UPDATE)", RFC 2136, April 1997.

[RFC 2535] Eastlake, D., "Domain Name System Security Extensions", RFC 2535, March 1999.

[RFC 3007] Wellington, B., et al., "Secure Domain Name System (DNS) Dynamic Update", RFC 3007, November 2000.

[RFC 3397] Aboba, B., and Cheshire, S., "Dynamic Host Configuration Protocol (DHCP) Domain Search Option", RFC 3397, November 2002.

[ZC] Williams, A., "Requirements for Automatic Configuration of IP Hosts", Internet-Draft (work in progress), draft-ietf-zeroconf-reqts-12.txt, September 2002.

[ZCHP] Guttman, E., "Zeroconf Host Profile Applicability Statement", Internet-Draft (work in progress), draft-ietf-zeroconf-host-prof-01.txt, July 2001.

23. Author's Addresses

Stuart Cheshire
Apple Computer, Inc.
1 Infinite Loop
Cupertino
California 95014
USA

Phone: +1 408 974 3207
EMail: rfc@stuartcheshire.org

Marc Krochmal
Apple Computer, Inc.
1 Infinite Loop
Cupertino
California 95014
USA

Phone: +1 408 974 4368
EMail: marc@apple.com